LAB 08 Shared application state: Provider and Cubit/BLoC

Unit 3 • 3-hour guided laboratory • CLO-5

Learning goal. Manage state shared between widgets with reactive observables and compare Provider's ChangeNotifier with flutter_bloc's Cubit.

Core concepts. Ephemeral vs app state; ChangeNotifier, context.watch/read, Cubit, BlocBuilder, immutable state and rebuild boundaries.

Before you start. Flutter app compiling from Lab 07; install packages separately for each task.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab08
cd mad_lab08
flutter pub add provider flutter_bloc
```

Solved task 1 | Shopping cart using Provider

Problem and objective. Share one cart model between a catalog and a summary area without threading callbacks through every widget.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() => runApp(ChangeNotifierProvider(
  create: (_) => CartModel(),
  child: const MaterialApp(home: ShopPage()),
));

class CartModel extends ChangeNotifier {
  final Map<String, int> quantities = {};
  int get count => quantities.values.fold(0, (a, b) => a + b);
  void add(String item) {
    quantities[item] = (quantities[item] ?? 0) + 1;
    notifyListeners();
  }
  void clear() { quantities.clear(); notifyListeners(); }
}

class ShopPage extends StatelessWidget {
  const ShopPage({super.key});
  @override
  Widget build(BuildContext context) {
    const items = ['Notebook', 'Marker', 'Pen', 'Folder'];
    final count = context.watch<CartModel>().count;
    return Scaffold(
      appBar: AppBar(title: Text('Campus shop • Cart: $count')),
      body: ListView(children: [
        for (final item in items)
          ListTile(title: Text(item),
            trailing: FilledButton(
              onPressed: () => context.read<CartModel>().add(item),
              child: const Text('Add'))),
        const Divider(),
        Consumer<CartModel>(builder: (_, cart, __) => Column(
          children: [
            for (final entry in cart.quantities.entries)
              Text('${entry.key}: ${entry.value}'),
            TextButton(onPressed: cart.clear, child: const Text('Clear cart')),
          ],
        )),
      ]),
    );
  }
}
```

How the solution works. The ChangeNotifier model stores shared mutable data and calls notifyListeners after each update. context.watch subscribes to state, while context.read invokes an action without subscribing. Consumer restricts one part of the tree to cart-specific rebuilds.

Expected output. Adding several items changes the cart count and per-item quantities; Clear empties it.

Check your work. Place a breakpoint in build and observe why listeners, not direct field changes alone, cause a rebuild.

Solved task 2 | Cubit: a predictable attendance counter

Problem and objective. Use flutter_bloc's Cubit to emit immutable integer states and update the UI with BlocBuilder.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

class AttendanceCubit extends Cubit<int> {
  AttendanceCubit() : super(0);
  void markPresent() => emit(state + 1);
  void reset() => emit(0);
}

void main() => runApp(BlocProvider(
  create: (_) => AttendanceCubit(),
  child: const MaterialApp(home: AttendancePage()),
));

class AttendancePage extends StatelessWidget {
  const AttendancePage({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Attendance • Cubit')),
    body: Center(child: Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        BlocBuilder<AttendanceCubit, int>(builder: (_, count) =>
          Text('Present students: $count',
            style: const TextStyle(fontSize: 24))),
        const SizedBox(height: 16),
        FilledButton(
          onPressed: () => context.read<AttendanceCubit>().markPresent(),
          child: const Text('Mark present'),
        ),
        TextButton(
          onPressed: () => context.read<AttendanceCubit>().reset(),
          child: const Text('Reset'),
        ),
      ],
    )),
  );
}
```

How the solution works. Cubit is a simpler member of the BLoC family: UI invokes methods, the Cubit emits a new state, and BlocBuilder redraws the consuming subtree. A full Bloc normally maps explicit events to states. Neither Cubit nor Provider inherently makes in-memory data persistent.

Expected output. Present count increments with each press and Reset returns it to zero.

Check your work. Explain why emitting state + 1 is easier to trace than mutating a shared global variable.

Scenario-based practice — unsolved

Scenario 1. Attendance across screens. Synchronize present count between a dashboard, class list and analytics page; decide between Provider or Cubit and justify state ownership.

Scenario 2. Shopping cart scalability. Model add/remove quantity, a coupon and asynchronous stock checks. Describe a test plan and which state should be immutable.

Technical viva — justify your reasoning

1. What does notifyListeners actually notify?
2. Why should widgets use read for button callbacks but watch when displaying derived data?
3. What distinction separates Cubit from a full event-driven Bloc?
4. What is a rebuild boundary and why does it matter?
5. When would you avoid global shared state entirely?
6. How does state survive a process restart?

Submission and self-check

Submit both runnable examples, a state-flow diagram and a comparison of Provider vs Cubit (3–5 sentences).